# Assessing Neural Network Workloads on Multicore Quantum Architectures

Sadit Joarder*, Daniel Kotlyar*, Rahaf Al Ostwani*, Justin Dang*, Ethan W. C. Wong*, Seham Al Abdul Wahid*, Farah Mohammadi*, and Arghavan Asad†

*Electrical, Computer and Biomedical Engineering Department, Toronto Metropolitan University, Toronto, Ontario, Canada
†School of Computer Science and Technology, Algoma University, Brampton, Ontario, Canada

***Abstract*—Quantum Computing suffers from problems in scalability. A proposed solution is Distributed Quantum Computing (DQC). By leveraging a multi-core quantum architecture, we can gain access to many more computational qubits. This paper uses qcomm, a simulator developed by Palesi et al. to show how DQC may be used to increase the number of available qubits without compromising on coherence or execution time. A neural network workload is shown on a mesh network with the use of WiNoC and without. Furthermore, other synthetic traffic patterns are studied to show which types of traffic are best suited on a distributed architecture.**

**Keywords— Distributed Quantum Computing (DQC), Convolutional Neural Networks (CNNs), Quantum Networks-on-Chip (QNoC), Machine Learning (ML) Accelerators, Quantum Simulator, High performance computing, Many-core Quantum Architectures.**

## I. Introduction

With the advent of widely accessible Large Language Models (LLM) such as ChatGPT or Deepseek, the world has shifted to more and more usage and abuse of these tools for everyday life. The greatest setback to these models is the time required to train them. For example, Deepseek takes approximately 3.7 days to train [1]. It's for this reason that maximizing efficiency remains such an important goal in the world of computation. The faster that complex and important workloads can be accomplished, the less power these workloads will consume and ultimately reduce their carbon footprint. This is the promise that quantum computing and all that study it offer. It's commonly stated that through use of quantum computing's parallelization capabilities, that an exponential speedup can be achieved over its classical contemporaries. Although it is true that quantum computing can offer exponential speedups, it is only for specific types of problems. The course of research has brought forward more and more problems that can be accelerated, most notably Shor's [2] and Grover's [3] works. The fact remains, however, that quantum computing has the power to revolutionise many of the systems powering the world; cryptography is an example [4]. The simultaneous rise of quantum technologies has evidently brought forth questions of whether it may be used to accelerate the training process of LLMs. While research suggests its viability [5], it also demonstrates the need for many more quantum computational bits for any semblance of usefulness. This paper seeks to show the viability of a quantum Network on Chip (NoC) to increase the number of available qubits without drastic compromises.

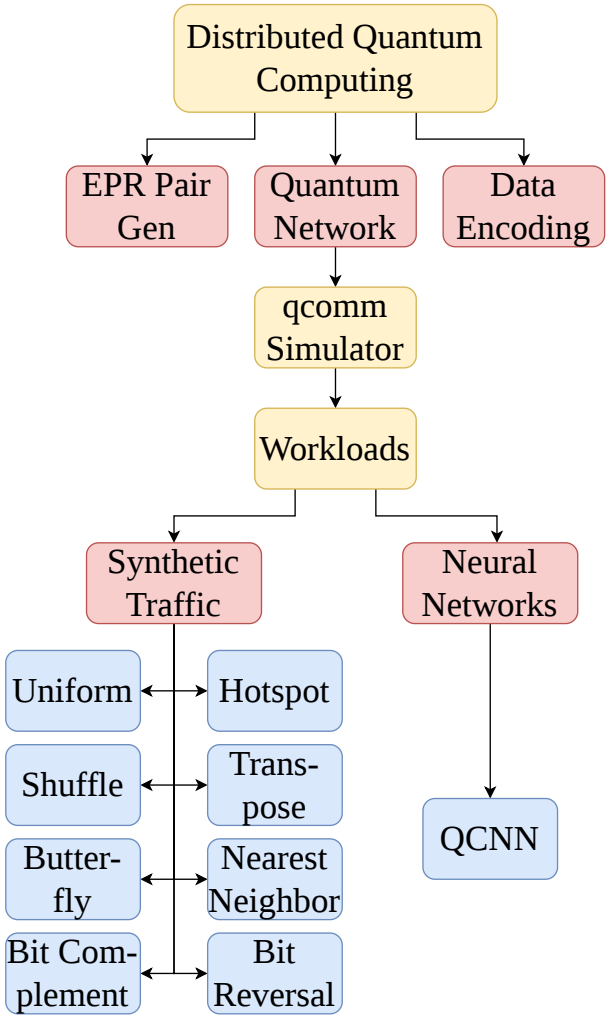



Fig. 1: Paper Overview

The biggest detriments to the viability of quantum computing are twofold. The era of quantum computing that we currently inhabit is known commonly as the Noisy Intermediate-Scale Quantum (NISQ) era, emphasizing the aforementioned issues. Noisy refers to information encoded within quantum bits (qubits) often being lost during computation. Intermediate-

scale refers to the relatively limited amount of qubits usable for computational tasks. As an example, to be used in an actual cryptographic setting, to factorize a 2048 bit integer using Shor's algorithm in 8 hours, 20 million qubits are required [6], which is a far cry from the 1000 that the IBM Condor offers [7].

Much research is conducted on addressing these issues. In terms of noise, error correction [8] or quantum memories [9] . In terms of speed and size of computations, neuromorphic architectures [10] and distributed quantum computing (DQC) [11] are being studied among many others. The last item mentioned is the most relevant for this paper. By leveraging DQC, we are able to give a quantum core access to many more qubits then a single core would be capable of. While distributed computing in a classical sense is generally done to improve execution speed by having multiple sub-processes happen in parallel, the main goal of our architecture is to be able to perform larger workloads in a quantum setting. Our chosen example of workload is a Quantum convolutional neural network (QCNN) with an input size of 10. While possible on current quantum processors, for the purposes of scalability, we seek to show the potential trade-off involved with switching from a monolithic structure to a distributed one. Using synthetic traffic patterns, we also show how certain traffics perform on this architecture to show its characteristics along with what types of workloads are best suited for such an environment.

All of the above is accomplished with the aid of "Qcomm", a simulation tool designed for quantum network on chip architectures [12]. Developed by Palesi et al. from the University of Catania, it is hugely important in the realization of this work.

The structure of the rest of the paper is as follows: Section II will talk about background information relating to concepts used in the paper. This includes explanations of multicore quantum architecture, quantum entanglement, quantum teleportation, routing, quantum convolutional neural networks and data encoding. Section III will briefly look at other works related to quantum networks on chip as well as QCNNs. Section IV will cover the different routing and communication protocols used in the realisation of qcomm. Section V will cover the way qcomm was setup for the experimentation. Section VI will discuss methods for evaluation of data as well as graphical results. Section VII & VIII will conclude and discusses areas where research could expand upon. Fig. 1 gives a general overview of the topics covered.

## II. Background

### A. Multi-core Quantum Architecture

Traditionally, classical computing tasks are performed on single chips. Given a set of processes to run, the computing tasks would sequentially perform each process as they come. As a result, in the efforts to increase speed, the idea of parallel processing emerged. The idea is to perform different tasks on different processors to allow them to be completed at the same or similar times. This is accomplished by having multiple cores and placing them in certain configurations (called "topologies") and having them communicate with each other [13]. This same idea can be applied to quantum computing as well, but not for the same reasons. Currently, quantum computing struggles due to a lack of available qubits [14], among other problems. While processing speed is still a concern [15], due to quantum computing being able to offer an exponential speedup over certain algorithms, workload size becomes the larger problem to solve.

As previously stated, the field of spreading quantum computing tasks is known as distributed quantum computing (DQC) [16]. This paper focuses specifically on quantum network on chip (qNoC). It is called a "network on chip" since the various cores communicate between each other to help alleviate tasks in terms of execution speed and size.

Of course, there exist other methods to improve the scalability of quantum computing. This is the reason for much research in DQC; as can be seen in [16]. In addition to NoC, it explains that computing tasks can be equally distributed between multiple quantum computers (known as a quantum farm) or even between multiple quantum farms. NoC remains the most attractive option, since quantum farms require qubit information to travel a larger physical distance [16], increasing thus the risk of noise and decoherence. However, the draw of multi-computer networks remains due to the higher possibility of hardware heterogeneity. Furthermore, other NoC paradigms may be suitable for DQC such as wireless NoC (WiNoC). WiNoC replaces the wired connections of traditional NoC with highspeed wireless connections in the form of ***** , this allows for improved transmission latency as congestion is avoided and every node is connected to each other, but it sacrifices power consumption [17]. Additionally, as seen in [11], various quantum hardware (e.g. ion traps, superconducting qubits, etc.) are not necessarily objective improvements over one another. By having multi-computer networks handle different parts of the same workload, the advantages of each hardware may be leveraged more effectively.

### B. Quantum Entanglement

As quantum mechanics and qubits fundamentally differ from classical computing, interconnections and communication between quantum nodes requires a paradigm shift. Unlike classical bits, qubits cannot be copied to another qubit [18]. Thus, one must exploit the properties of maximally entangled qubits to transfer data from one remote quantum core to another.

In a quantum network, EPR pair generation is typically performed under the Barrett-Kok protocol which exploits matter qubits (e.g., trapped ions, superconducting qubits, electron spin) and linear optics to entangle them. These matter qubits store quantum information, otherwise known as a quantum memory, or simply, a qubit.

The process begins by initializing each qubit to the superposition state:

$$|+\rangle = \frac{1}{\sqrt{2}}\left(|\uparrow\rangle + |\downarrow\rangle\right)$$

where $|\uparrow\rangle$ is the $|0\rangle$ qubit state and $|\downarrow\rangle$ is the $|1\rangle$ qubit state. These can also be understood as the up and down spin of an electron.

Each qubit within a node is embedded within an optical cavity. Using electron spin as an example, when an optical pi-pulse is applied to this optical cavity, the qubit absorbs the incoming photon and gains energy [19]. This consequently makes the qubit either: jump from the ground state $|\downarrow\rangle$ to a higher, excited state $|e\rangle$, or do nothing $|(\uparrow\rangle$ to $|\uparrow\rangle)$. In the case of the down-spin, naturally, the qubit will want to return to the ground state, effectively emitting a photon carrying the information of the qubit's state. In the case of the up-spin, the qubit will emit nothing.

Two qubits A and B emitting a photon from separate nodes can be represented as a joint state of probabilities:

$$|+\rangle_A \otimes |+\rangle_B = \frac{1}{2}\left(|\uparrow\uparrow\rangle + |\uparrow\downarrow\rangle + |\downarrow\uparrow\rangle + |\downarrow\downarrow\rangle\right)$$

Through an optical channel, this emitted photon travels to one of two inputs of a 50:50 beam splitter, where it is simultaneously met with another photon coming from a different node, as seen in Fig. 2. If only one of the two qubits emitted a photon, there are two unverified possibilities of where the photon could have come from according to the principle of superposition in quantum mechanics – qubit A or qubit B. This theoretically makes the path of where the photon came from ambiguous, regardless of which of the two input ports at the beam splitter it entered.

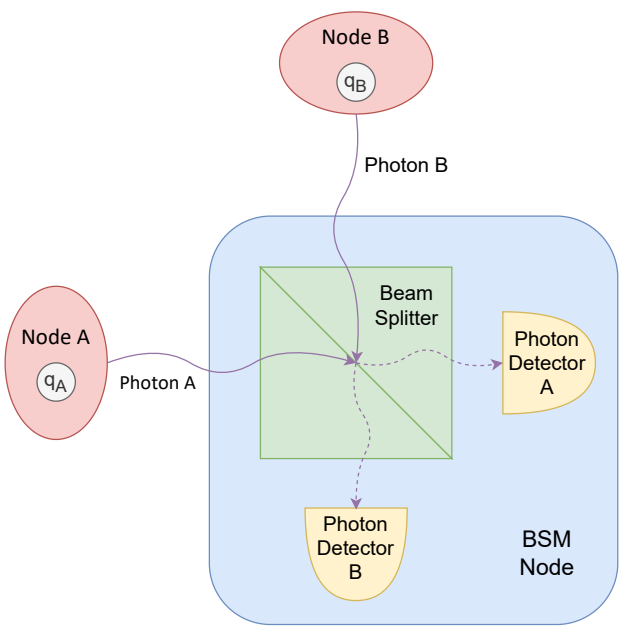



Fig. 2: Barret-Kok EPR generation protocol

In the 50:50 beam splitter, there are four scenarios of how the photon(s) could navigate:

1) Both photons are reflected or both photons are transmitted, causing destructive interference from a $\pi/2$ phase shift in the beam splitter where nothing arrives at either of the detectors.

2) One photon is reflected while the other is transmitted, causing constructive interference from no phase shift in the beam splitter where both photons arrive at either of the detectors.

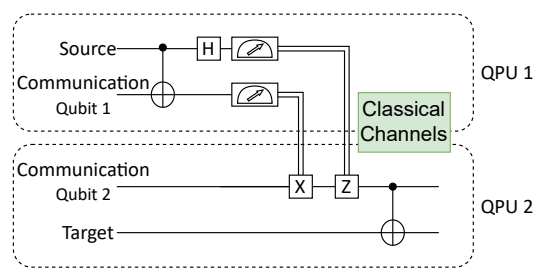
3) Only one photon arrives at the beam splitter (reflected or transmitted), causing only one photon to be read by either of the detectors.

4) Neither photon arrives at the beam splitter, causing nothing to be read by either of the detectors.

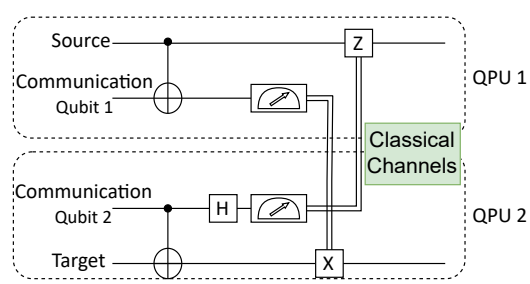
If only one photon arrives at either of the detectors, it is a success. Otherwise the process has to be restarted. Another round is performed where an $X$-gate is applied to both qubits, effectively flipping the up- and down-spin states. This second round ensures that both the $|\uparrow\downarrow\rangle$ and $|\downarrow\uparrow\rangle$ probabilities are tested and that the final state is pure/maximally entangled. Both rounds must be successful for the pair of qubits to be maximally entangled [19].

### C. Quantum Teleportation

Quantum teleportation is a phenomenon which allows for maximally entangled qubits to transmit information from one remote node to another without physically transferring the particle [16]. Once a pre-shared EPR pair exists, one can choose between two protocols to apply a remote operation, Teledata and Telegate.




(a) Teledata




(b) Telegate

Fig. 3: Common teleportation protocols

*1) Teledata:* A Bell State Measurement (BSM) must be performed to deliver the data from one source to another. BSM consists of a CNOT gate which has the source qubit as the control and the entangled qubit as the target. Afterwards, a Hadamard gate is applied to the source qubit and both qubits are measured [16]. The measurement is necessary to correct any phase or bit flip errors which may have occurred during the process. Decoding the measurements and solving any bit flip or phase error provides the destination QPU with the quantum state of the source qubit [20].

*2) Telegate:* Unlike Teledata, Telegate does not transfer the exact quantum state from one core to another. The protocol enables remote operations such as a CNOT gate between qubits from two remote cores. With the pre-shared EPR pairs, the controlled gate must be applied to the source's qubits and the destination's qubits [16]. This allows the source data qubit to act as the control for the destination data qubit. Finally, the communication qubits are measured and decoded to detect any errors bit flip or phase flip errors during the process.

### D. Quantum CNN

A Quantum CNN is as the name suggests the quantum implementation of a convolutional neural network. The QCNN is based on the regular CNN architecture utilizing a similar architecture of repeating convolutional and pooling layers until an eventual fully connected layer is reached.

*1) Convolutional Layer:* This is the main layer within the CNN responsible for the actual feature extraction and is the main layer of the architecture [21]. This layer calculates and stores the dot product of the input image by sliding a filter over the image. The final result is known as the feature map. In order to avoid overly complex results the convolutional layer is not usually connected to the entire input all at once [21] and the result of the convolutional layer is known as a feature map which is classically represented as a square matrix. The convolutional Layer additionally has 3 additional parameters which can be used to significantly reduce the complexity of the output [22].

a) *Depth:* This refers to the depth of the output volume produced by the convolutional layer. Alternatively the depth can be thought of as the number filters [23]. This parameter can be set manually based on the number of neurons within the layer. Reducing this parameter reduces the number of neurons within the network but can also heavily reduce the pattern recognition effectiveness of the model [22].
b) *Stride:* The stride is essentially how many pixels the input filter moves over the input matrix [22]. Small strides means the receptive fields of the filter overlap many times over creating very large activations. A greater stride reduces this aforementioned overlapping and will produce a result of lower spatial dimensionality.
c) *Padding:* Padding involves adding a border to the kernel filter. The simplest method for this is to padding the filter with zeros. The main thing to keep in mind with padding is that it alters the spatial dimensionality of the convolutional layer's output [22]. This is represented in the following formula where V represents the input volume size, R represents the receptive field size, Z represents the amount of the zero-padding set, and S represents the stride [22].

$$\text{Output size} = \frac{(V - R) + 2Z}{S} + 1$$

*2) Pooling Layer:* The main purpose of the pooling layer is to reduce the complexity of the model [21]. In quantum terms this means reducing the number of qubits in order to reduce computational costs of the implementation [24]. The most common method for achieving this is known as max pooling. This uses a 2 x 2 kernel with a stride of 2 in order to scale down the activation map by 25 [22]. Since the pooling layer is destructive, having larger kernels for the pooling layer will usually greatly reduce the performance of the model [22].

*3) Fully Connected Layer:* The fully connected layer is essentially responsible for classifying the image. This layer receives feature maps from the pooling layer, these feature maps are then compared to the originally computed feature maps and performance loss classifies the data within the image [21]. The overall structure of the CNN design can be seen in Figure 4.

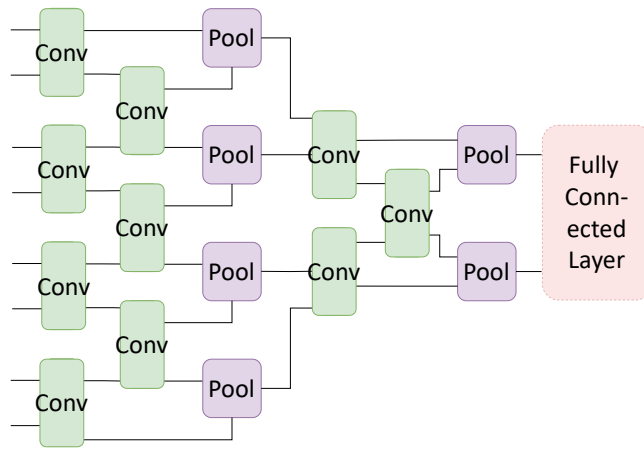



Fig. 4: Overview of CNN design

### E. Data Encoding

Quantum computing is primarily performed on a hybrid classical-quantum platform. Hence, classical data points must be encoded into quantum states before being inputted into a quantum network. The three most common encoding methods are basis, angle, and amplitude encoding [25].

*1) Basis Encoding:* In basis encoding, the classical data point is first converted from decimal to binary. Then, each bit in the binary string is mapped to an unused qubit of a single-qubit basis state: $|0\rangle$ or $|1\rangle$. The result of the encoded classical data point becomes the tensor product of these quantum states, creating a computational basis state [26]. Because of this, basis encoding is the simplest to implement. However, it comes at the cost of space inefficiency and thus, decreased qubit coherence since one qubit is required for every bit of a classical data point. This also makes it practical only for discrete data type applications [25] and when the classical data input size is small. Below is the possible final quantum state of a basis-encoded binary string, where $|\psi\rangle$ is the new final state as a result of the tensor product between every encoded state of the binary string $|b\rangle$,

$$|x\rangle = |b_1\rangle \otimes |b_2\rangle \otimes \cdots \otimes |b_n\rangle = |b_1 b_2 ... b_n\rangle$$

*2) Angle Encoding:* Also referred to as phase or rotation encoding, this method takes in a classical data point consisting of a real number and converts it to a phase angle. Using this, a Pauli-X, -Y, or -Z rotation gate is multiplied by the ground state $|0\rangle$ of an unused qubit [26]. Because of this, angle encoding is best applied to continuous data types. It also only requires one qubit per classical data point [25], which makes it far more efficient than basis encoding. Below are the possible final quantum states of an angle-encoded value, where $|\psi\rangle$ is the qubit's final state; $R_X()$, $R_Y()$, and $R_Z()$ are the Pauli-X, -Y, and -Z rotation gates; $\theta$ is the angle, and $|0\rangle$ is the ground state,

$$|\psi\rangle = R_X(\theta)|0\rangle = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) \end{bmatrix} = \cos\left(\frac{\theta}{2}\right)|0\rangle - i\sin\left(\frac{\theta}{2}\right)|1\rangle$$

$$|\psi\rangle = R_Y(\theta)|0\rangle = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) \end{bmatrix} = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle$$

$$|\psi\rangle = R_Z(\theta)|0\rangle = \begin{bmatrix} e^{-i\theta/2} \\ 0 \end{bmatrix} = e^{-i\theta/2}|0\rangle$$

such that,

$$R_X(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$$

$$R_Y(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$$

$$R_Z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}$$

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

*3) Amplitude Encoding:* In amplitude encoding, all classical data points of the input – consisting of real numbers, are combined into a single one-dimensional vector. Each value in the vector is multiplied by the square-normalization of the vector, each representing a unique probability amplitude for a corresponding basis state of the qubit(s) used [26]. This makes amplitude encoding extremely qubit-efficient because only a $\log_2 n$ amount of qubits is required to encode n bits. Thus, this method is best used for continuous data types [25] and large classical datasets/inputs. However, its complexity and lack of a concrete circuit pattern when varying the input parameters serves a challenge for implementing it into quantum networks. Below is the possible final quantum state of an amplitude-encoded dataset of values, where $|\psi\rangle$ is the dataset's final state, $x$ is a value in the dataset, and $|n-1\rangle$ is the integer representation of each basis state of the qubits used,

$$|\psi\rangle = \frac{x_1}{\sqrt{\sum_{i=1}^{n} x_i^2}}|0\rangle + \frac{x_2}{\sqrt{\sum_{i=1}^{n} x_i^2}}|1\rangle + \cdots + \frac{x_n}{\sqrt{\sum_{i=1}^{n} x_i^2}}|n-1\rangle$$

## III. Related work

While other publications discuss quantum traffic and quantum neural networks; to our knowledge, none exist that talk about testing such workloads on a quantum NoC. We will briefly look at related works in both the fields of qNoCs and QCNNs.

*1) Quantum Network on Chip:* Lauded as a possible solution for the scalability issue that quantum computing faces [14], quantum NoC is a major topic, as it is the basis for all conclusions in this paper. They are performed using a simulator known as qcomm [12], created by Palesi et al. It introduces a simulation tool to assess various communication metrics for any given quantum circuit. The work assesses communication times using only one traffic pattern, that being the "uniform" pattern. [27] is one of the few other publications that discusses a quantum NoC. However, it does not comment on uses or applications. Instead, it discusses the difficulties of a physical implementation as well as the different quantum hardware that may be used for an implementation of a quantum NoC. Similarly, [14], [28], [29] and [30] also investigates possible implementations of quantum NoCs, but focuses more on hardware implementations.

*2) Quantum CNN:* The main workload evaluated in this paper, in addition to traffic patterns, is the quantum convolutional neural network (QCNN). This is mainly due to the fact that there exists a tested and working quantum circuit for it [24]. The aforementioned article, produced by IBM's Qiskit machine learning development team, is the basis for the QCNN simulated in this paper. It is based on an initial proposition of a QCNN [31] by Cong et al.

There exist other proposed implementations of QCNNs, one such using a degressive circuit [32], another mixes both amplitude and angle encoding [33] and another doing something similar by introducing a new encoding process to lower the number of qubits required [34]. These other implementations of QCNNs are all valid and may equally be chosen to simulate on qcomm.

It is important to note that despite Deep Neural Networks (DNN) being more versatile and understood within a classical context [37, 38, 37], quantum implementations of DNNs are much less understood and suffer from problems such as the "vanishing gradient problem"[38]. Circuits for DNNs are complex and not as easily to simulate. Thus, the decision was made not to do so for this paper.

## IV. Communication Platform for Multi-Core Quantum Architecture

### A. Routing

*1) Routing Protocols:* As seen in the prior section, quantum teleportation protocols often send classical data between the two cores as part of an error correction process. For this, a classical routing algorithm must be used to transmit the information from one to another. One such example, along with the algorithm used for the simulations later is the "XY" routing algorithm. Simply put, the information travels along the necessary neighboring cores on the horizontal plane, then it travels along the vertical neighboring cores[39]; as can be

found in Fig. 5. The advantage of the XY routing algorithm is its simplicity. Without the need for complicated instructions compared to other routing algorithms, XY is much easier and more lightweight to implement.

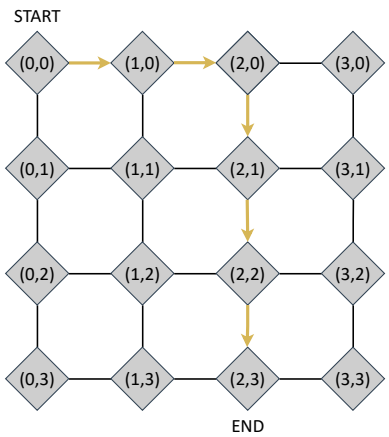



Fig. 5: XY Routing Protocol

*2) Router:* As seen in Fig. 7, the generic router architecture consists of various components such as the crossbar unit, input/output ports, and lastly Router computation (RC), Virtual Channel Allocation (VA), and Switch allocation (SA) units. The data path is made up of the ports and crossbar unit while the control path is made up of the RC, VA and SA units [40]. The four stage pipeline of the router is made up of the RC, VA, SA and finally crossbar unit (XB stage). The RC stage employs a routing algorithm in order to calculate the final output port. The VA stage determines what unused downstream virtual channel (VC) will be used. The SA stage accepts the flits within the VC into the input ports of the crossbar. Finally during the XB stage the crossbar directs the flits towards their destination[40].

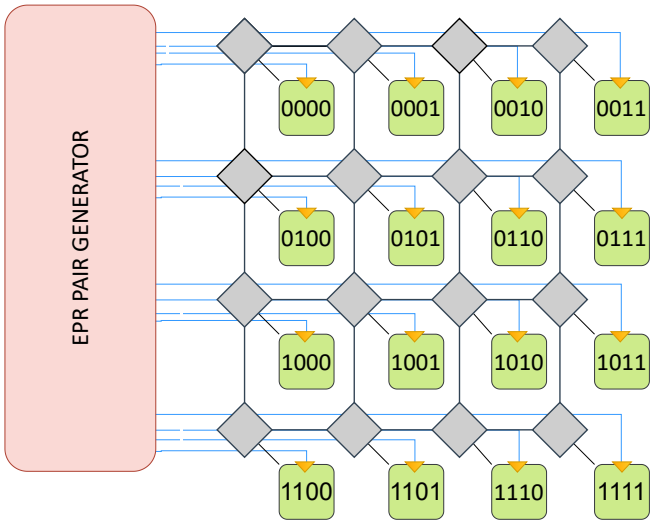
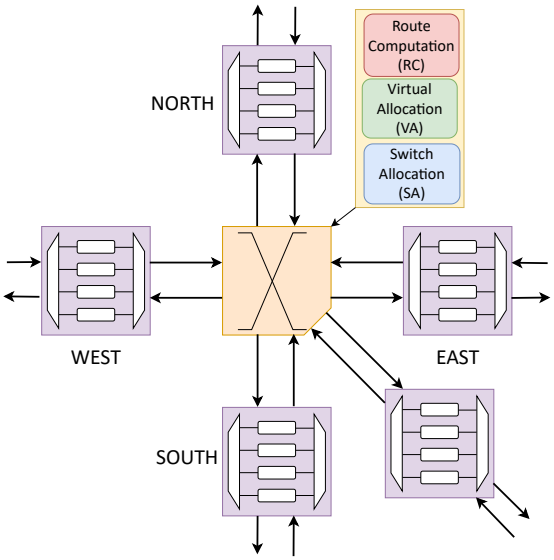
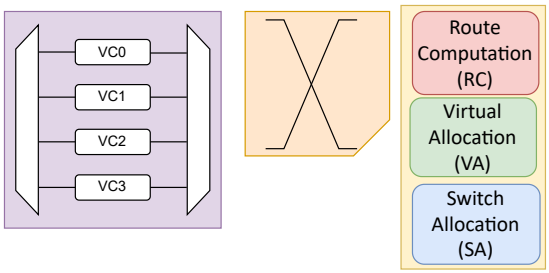



Fig. 6: Mesh Architecture employed by QCOMM

*3) Connections:*




(a) Overview of Router Architecture




(b) Router components. From left to right, port, crossbar, and pipeline

Fig. 7: Common Router Design

## V. QCOMM SETUP

This paper utilizes Qcomm, a simulation tool, to evaluate the impact of communication under various traffic workload conditions on multi-core quantum architectures [12]. Qcomm was chosen for its flexibility in modeling quantum NoC architectures and its support for configurable system-level parameters, which are summarized in Tables I and II.

TABLE I: QCOMM Simulator Architecture

| Parameter | Value |
|---|---|
| NoC Topology | 4×4 2D mesh |
| Qubits per Core | 10 |
| Routing Algorithm | XY routing |
| LTM Port per Core | 2 |
| Link Width | 8 bits |
| Mapping Type | Sequential/Random |

In addition to configurable architecture and parameter files, a circuit description file must be provided to Qcomm. A circuit file consists of several time slices in which certain (Pauli XYZ,

H, CNOT, etc.) can be defined. Each gate is represented as a tuple with numbers indicating the qubits involved in the operation, along with the name of the gate in front.

For the purposes of experimentation, the simulator is set up in a 4 X 4 mesh configuration. This means that the network is composed of 16 cores. Each quantum core is assigned 1 NoC router, which transfers classical data from one core to another. This is important as it relates to qcomm's choice on quantum communication protocol, Teledata. To perform Teledata, each core must have a pair of pre-shared, maximally entangled qubits. For this purpose, each core has a certain number of light to matter (LTM) ports which send qubits to an EPR generator. That generator then entangles two qubits at different cores as needed.

For the purposes of entanglement, after two qubits have been entangled, classical data must also be sent between the two cores at which the qubits reside. This data is sent in the form of a "packet". The packet size in bits is determined with the following equation [12]:

$$2[log_2(nqubits)] + 2$$

The packet contains the addresses to the source and destination qubits in which *nqubits* is defined by the number of total qubits in the system. The additional 2 bits represent the bits measured during Teledata for error correction at the destination core [12]. Pictured in Figure 7 is a possible example of how information may be distributed within a packet

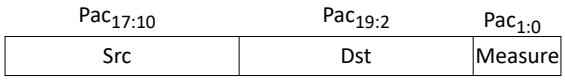



Fig. 8: Barret-Kok EPR generation protocol

TABLE II: QCOMM Simulator Parameters

| Parameter | Value |
|---|---|
| Gate Delay | 15 ns |
| EPR Delay | 1 µs |
| Distillation Delay | 0.01 ns |
| Pre-Processing Delay | 390 ns |
| Post-Processing Delay | 30 ns |
| Wbit Rate | 12 Gbps |
| Token Pass Time | 1 ns |

## VI. Experimental Evaluation

### A. Effects of Bottlenecking Synthetic Traffic Patterns

Using Qcomm's default parameters, three trials of every synthetic traffic pattern were simulated while varying for 200, 400, 600, 800, and 1000 gates. This is referred to as the zero bottleneck data. To bottleneck the classical aspect of the multi-core system, the quantum parameters were improved by 20 times. This includes dividing EPR generation, EPR distribution, pre-processing, and post-processing delay times by 20 in Qcomm. The same three trials were simulated while varying for the same gate amounts. To bottleneck the quantum

TABLE III: Summary of Traffic Patterns Used in Simulation

| Traffic Pattern | Description | Real-World Relevance |
|---|---|---|
| Uniform | Traffic sent randomly to all nodes | Baseline for evaluating load balancing |
| Bit Complement | Destination is bitwise complement of source | Stresses longest communication paths |
| Transpose | Communication with transposed coordinates | Used in matrix transposition operations |
| Bit Shuffle | Source bits are circularly left-shifted | Tests long-distance interconnect performance |
| Bit Reversal | Bits of source address are reversed | Simulates irregular routing patterns |
| Hotspot | Majority of traffic targets a few nodes | Models network congestion and thermal hotspots |
| Neighbour | Traffic limited to adjacent nodes | Simulates localized communication like nearest-neighbour search |
| Butterfly | Structured multi-hop routing pattern | Found in FFT and deep neural network layers |

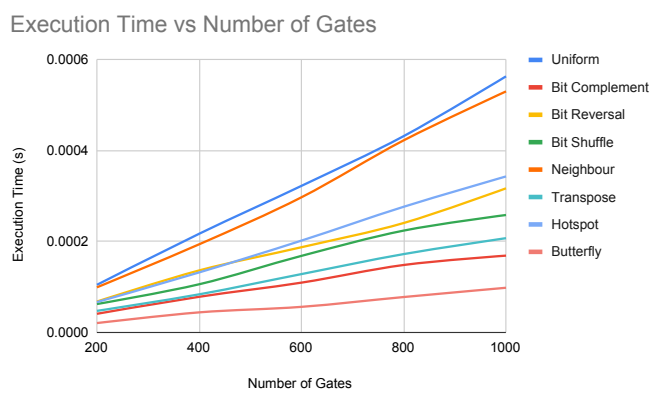



Fig. 9: Execution Time vs Number of Gates of all 8 synthetic traffic patterns under no bottleneck conditions

aspect of the multi-core system, the classical parameters were improved by 20 times. This includes multiplying memory bandwidth by 20, and dividing decode time per instruction and NoC clock time by 20. The averages of these three trials for each bottleneck condition were averaged and results on execution time and qubit coherence were graphed against the number of gates for every traffic pattern.

Several observations were made when comparing the graphs of these three bottleneck conditions. The analysis specifically focuses on the relative change in behaviour of the synthetic traffic patterns as quantum parameters improve – i.e., as quantum computers get faster. The general trend is that the execution times and qubit coherences of all traffics become closer in measurement to one , as seen in Fig. 12 and 13 in comparison to Fig. 9, 11, 14, and 15.

Neighbour goes from yielding similar results to becoming better than uniform as quantum parameters improve. However, out of all traffic patterns, these two by far perform the worst

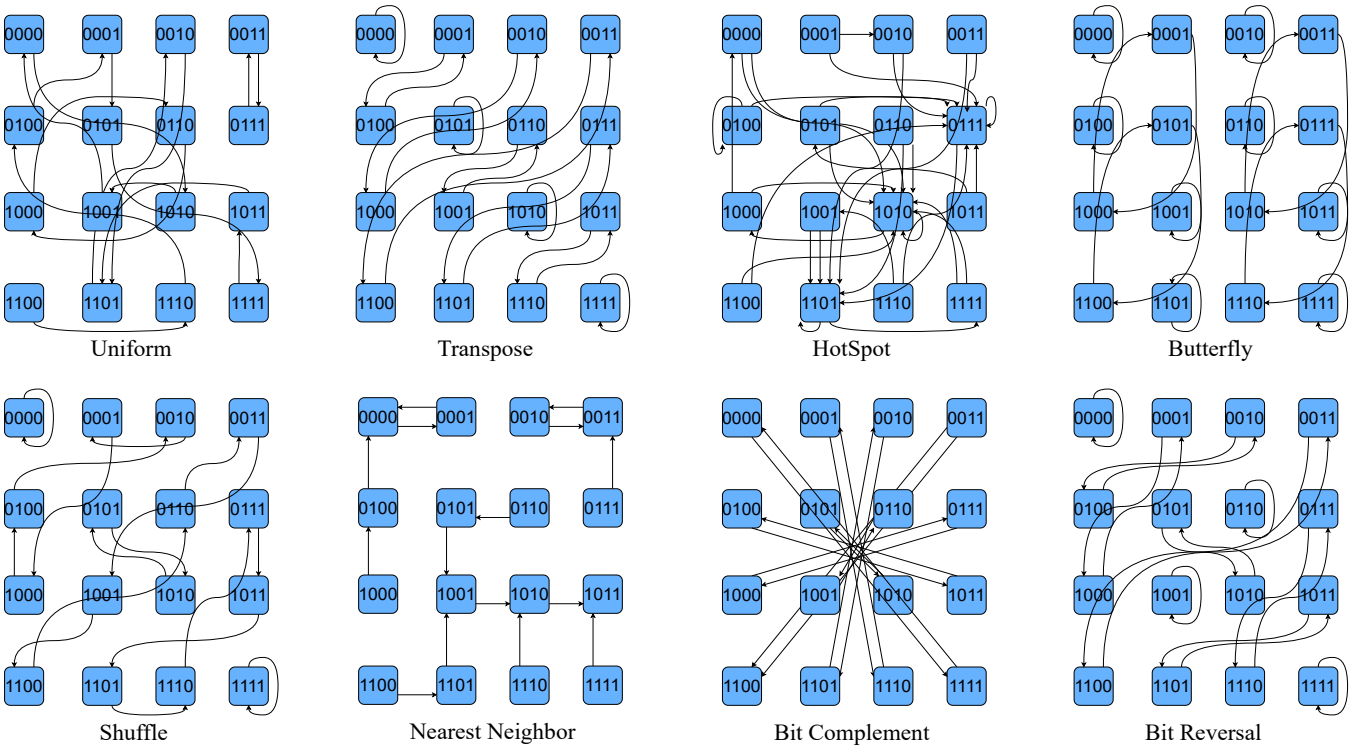



Fig. 10: Intercore Communications of Synthetic Traffic Patterns

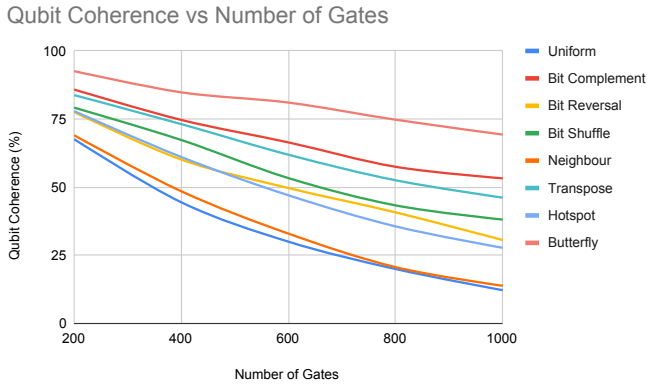



Fig. 11: Qubit Coherence vs Number of Gates of all 8 synthetic traffic patterns under no bottleneck conditions

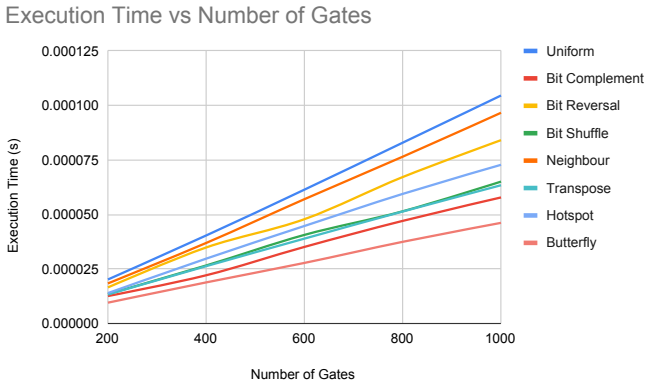



Fig. 12: Execution Time vs Number of Gates of all 8 synthetic traffic patterns when quantum parameters are improved by 20 times

across all bottleneck conditions, whereas butterfly always yields the best results. As for the traffics in between, the execution time and qubit coherence of bit shuffle gradually improves to match the performance of transpose. It can also be observed that hotspot goes from performing worse than bit reversal to performing better.

These results first and most importantly suggest that a quantum network communicating in a butterfly pattern should produce the lowest execution times and highest qubit coherence results. The data also reveals the massive potential of hotspot to outperform other patterns in this domain as quantum computers get faster, with bit shuffle also showing potential, but less. This is visualized in Fig. 16 and 17 – where hotspot goes from 6% worse to 7% better that bit reversal in execution time improvement relative to uniform, and 17% worse to 3% better in qubit coherence.

EPR generation takes up a large share of execution time under typical, zero-bottleneck conditions. When quantum parameters improve, the net effect of EPR generation on total execution time drastically decreases, which overall narrows the difference in performance between the patterns. The improvement in certain patterns relative to others is for the same reason. Patterns that spread out traffic across longer distances naturally perform worse since classical factors (e.g., routing path distance and congestion) become the bottleneck. From the

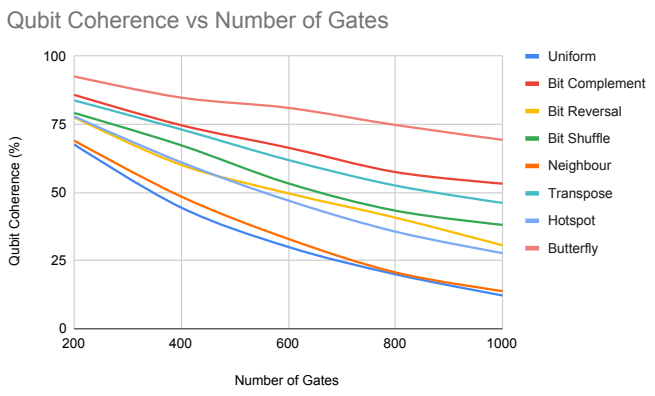



Fig. 13: Qubit Coherence vs Number of Gates of all 8 synthetic traffic patterns when quantum parameters are improved by 20 times

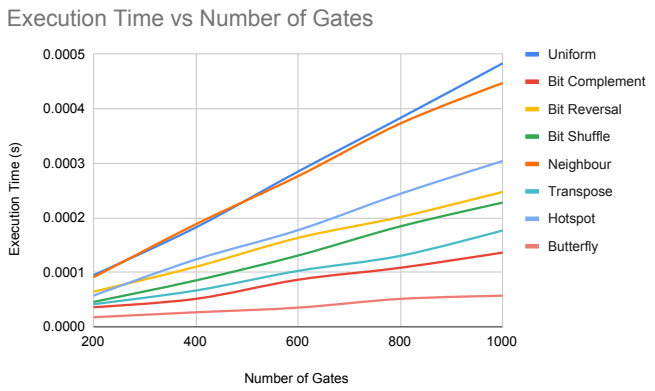



Fig. 14: Execution Time vs Number of Gates of all 8 synthetic traffic patterns when classical parameters are improved by 20 times

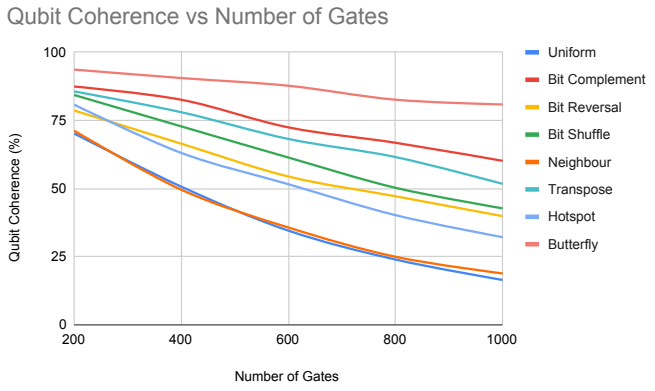



Fig. 15: Qubit Coherence vs Number of Gates of all 8 synthetic traffic patterns when classical parameters are improved by 20 times

perspective of the patterns that communicate more exclusively within the same core like hotspot, these perform better since they minimize the need for classical routing. The results

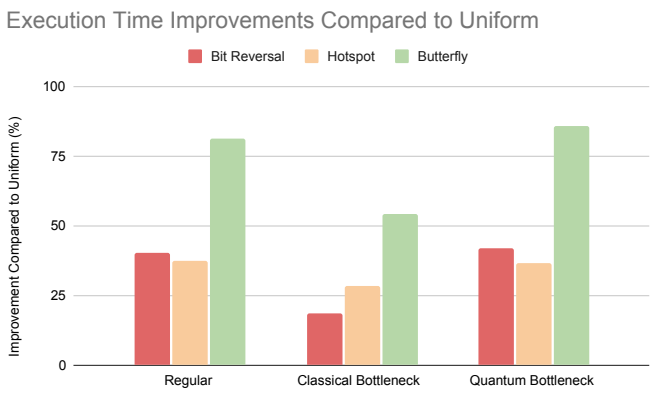



Fig. 16: Percentage decrease in execution time of bit reversal, hotspot, and butterfly relative to uniform under zero, classical, and quantum bottleneck conditions

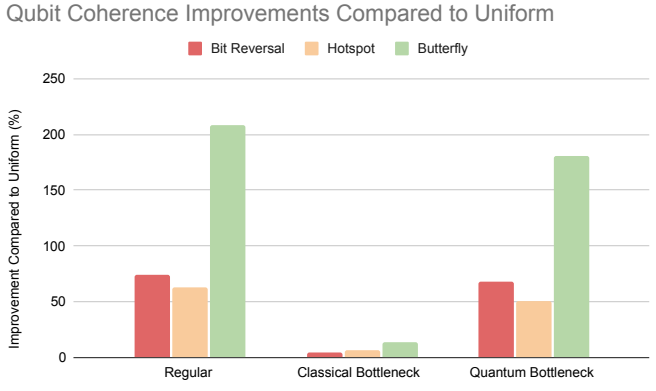


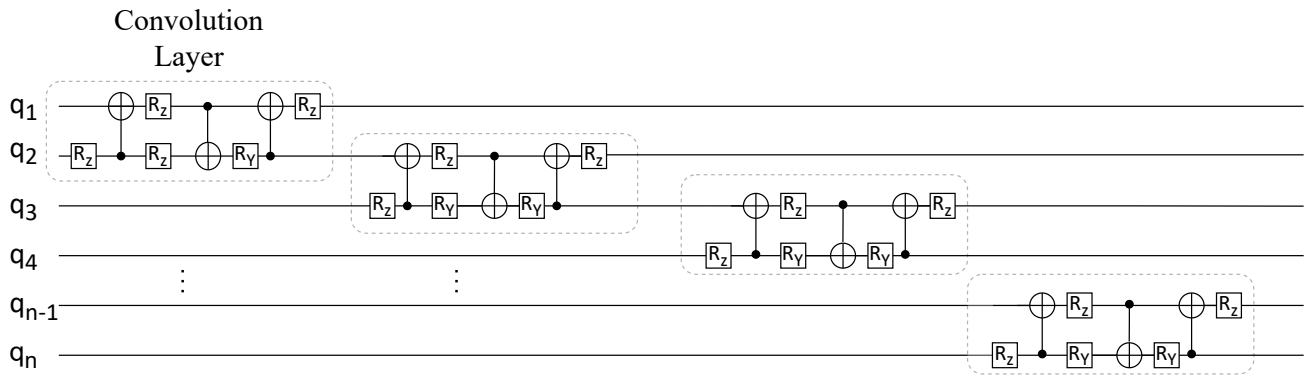

Fig. 17: Percentage increase in qubit coherence of bit reversal, hotspot, and butterfly relative to uniform under zero, classical, and quantum bottleneck conditions

showcase that as quantum distributed platforms speed up, they are better suited to handle memory intensive applications which current monolithic quantum computers struggle with.

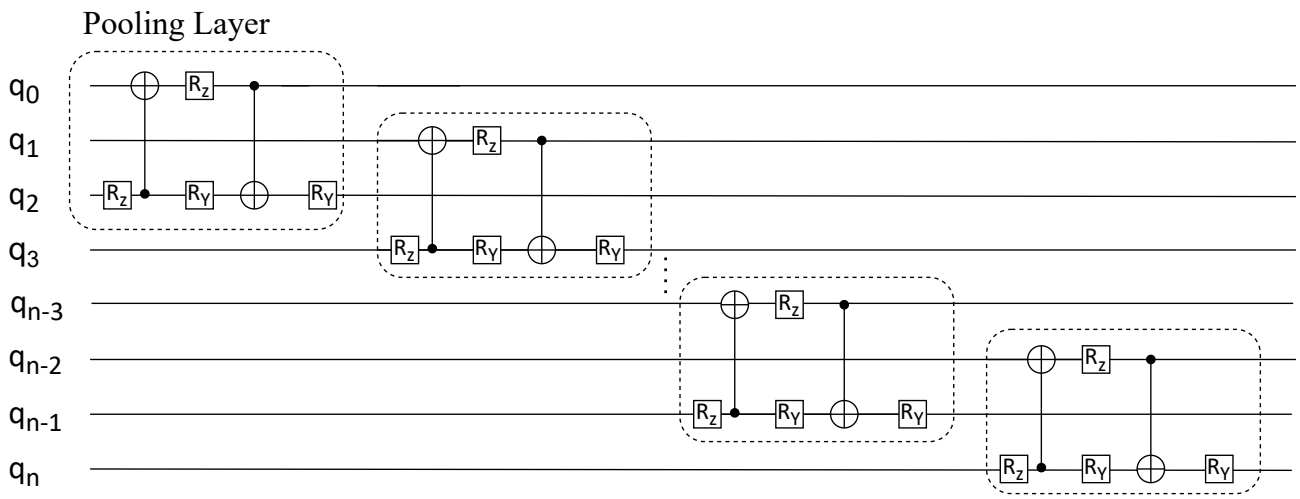
### *B. Monolithic vs Distributed QCNN*

The NN workload tested was a QCNN. The QCNN, which functions similarly to the regular CNN, leverages the power of quantum to perform computations faster. In order to analyze the performance of a QCNN on DQC using qcomm, a circuit representation was needed in order to feed the simulator. A version of the parametrized circuits representing the convolutional layer and pooling layer provided by IBM [25], were used. The resultant circuits can be seen in Fig. 18.

The circuits were generated using python and simulated using qcomm. Due to the proposed architecture of 100 logical qubits, the input size was incrementally increased from 2 to 10, with an input size of 10 utilising all qubits. Along with quantum neural network workloads, this study uses the ZFeatureMap angle encoding implementation found in Qiskit's circuit library as described in Fig. 19. Using another program developed by Palesi et al., qasm2qcomm, any simple Qiskit




(a) CNN Convolutional Layer




(b) CNN Pooling Layer

Fig. 18: CNN components: (a) Convolutional layer and (b) Pooling layer.

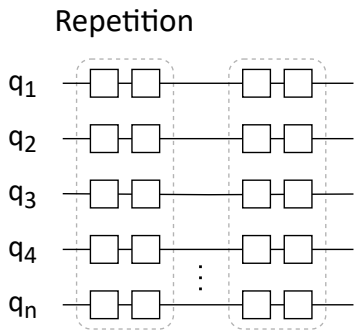



Fig. 19: Qiskit's ZFeatureMap Circuit

circuit [41] can be made into a qcomm-compatible circuit. The ZFeature map's simple usage of only one-qubit gates and two time slices, makes implementing it into the QNN circuit files easier, all while maximizing qubit coherence.

Three trials were recorded at each input size to obtain accurate and precise results. The mesh network used for the experiments mostly consisted of the default parameters and architecture as shown in Table I, II and III, with an exception of the number of ports. To leverage the benefits of parallelization within a distributed architecture, 3 LTM ports were used to allow entanglement generation between multiple cores simultaneously. Beyond 3 LTM ports, its effects begin to decrease and yield marginal improvements [12]. Along with wired NoC, WiNoC was enabled to showcase the results of the distributed architecture without the issue of traffic congestion in the classical channels as shown by the synthetic traffic patterns. Finally, these results will be compared to a monolithic QPU with 160 qubits and default parameters to find areas where the NoC architecture could offer an improvement over a monolithic one when running a QCNN workload

The main parameters we compared were execution time and qubit coherence, as can be seen in Fig. 20 & Fig. 21.

As seen via the following data, in general monolithic quantum processors generally outperform distributed architectures with wired NoC by 7.04% in coherence and 86.37% in execution time. As the main contributor to this is the necessity for large amounts of classical communication. The slower

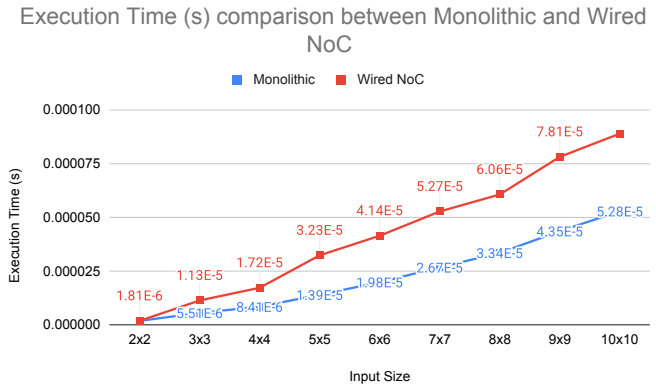



Fig. 20: Execution Time vs Input Size between Wired NoC and Monolithic architecture

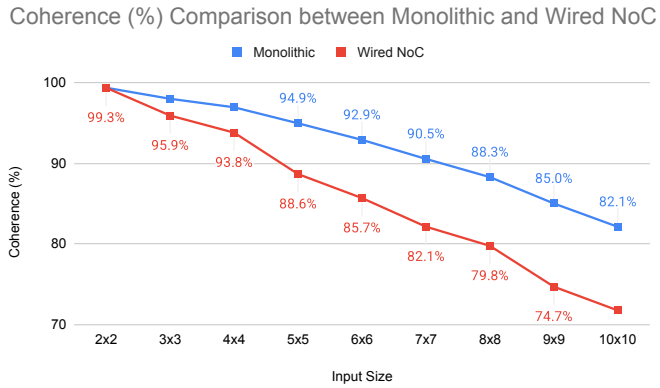



Fig. 21: Coherence (%) vs Input Size between Wired NoC and Monolithic architecture

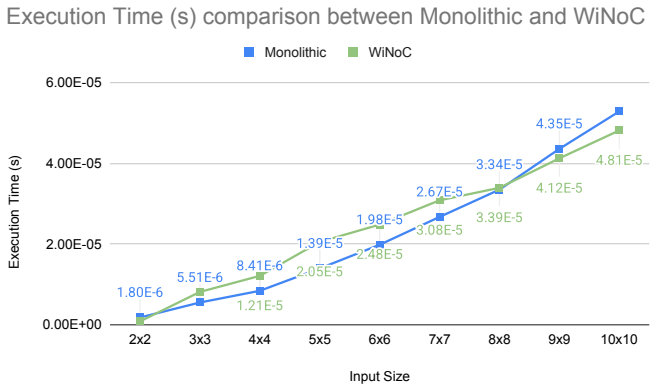



Fig. 22: Execution Time vs Input Size between WiNoC and Monolithic architecture

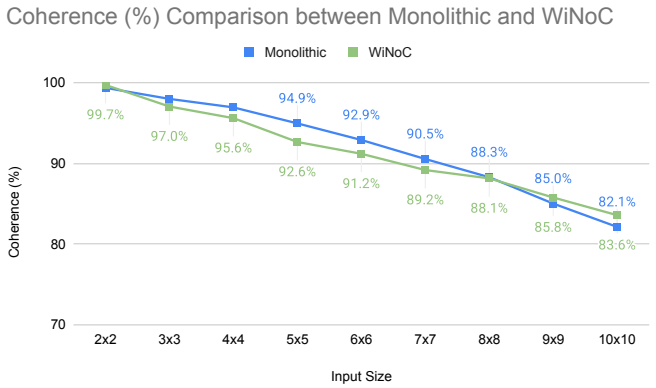



Fig. 23: Coherence (%) vs Input Size between WiNoC and Monolithic architecture

execution time is mainly a result of classical communication being subject to larger amounts of congestion and the need for EPR pair generation which also impacts coherence. Factoring in these disadvantages of distributed quantum, with current technologies a distributed quantum design relying on classical communication performs worse in both coherence and execution time. To improve the performance of the distributed architecture, wireless communication can be used to reduce latency within the system.

When switching to a WiNoC communication protocol, the distributed system outperforms the monolithic system in terms of both execution time and coherence when running a QCNN workload. However, this only occurs once the input size has passed a certain number. The WiNoC system begins to outperform the monolithic system in both coherence and execution time, starting at a 9x9 input image size. For the aforementioned input size, the execution time is 5.37% better on the WiNoC. By the time a 10x10 input image size is reached, the execution time is 8.91% better on the WiNoC, suggesting a improvement over monolithic for increasing input size which is supported by the results found in the research by Lit et al [42].

For coherence a 9x9 input size on the WiNoC design achieves a 0.75% improvement over the monolithic design and for a 10x10 input size the WiNoC achieves a 1.45% improvement over the monolithic design. This suggests that the coherence of a distributed WiNoC system is impacted less for increasing input sizes over a designated input size.

As input sizes grow larger, the benefits of parallel computations allow the distributed WiNoC architecture to outperform the monolithic architecture. Fig 24 & 25, demonstrate the importance of LTM ports and parallelization for a distributed architecture. WiNoC architecture begins to provide better coherence and executions with a saturation in performance at 4 LTM ports.

The results here show that a WiNoC design performs better than a monolithic design for larger input size and is less negatively affected by increasing input sizes. This makes the distributed WiNoC system ideal for large input sizes showing how the design accomplishes the explicit goal of a distributed quantum system which is to scale a quantum computer.

## VII. Future Directions

### A. Limitations of Quantum Computing

While the adoption of multi-core quantum architectures offers a promising path toward scalable quantum computing,

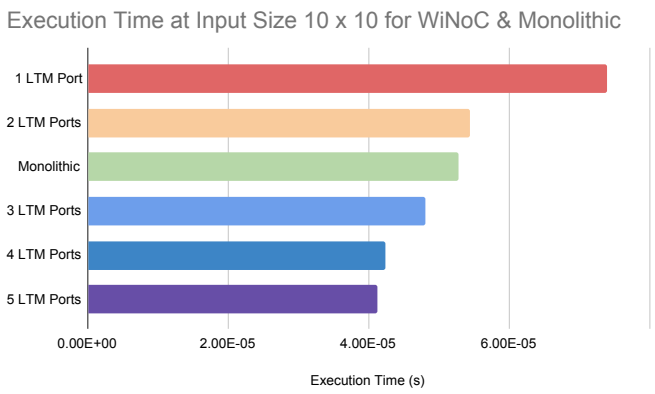



Fig. 24: Difference in Execution Time for WiNoC with Varying LTM Ports and Monolithic Architecture

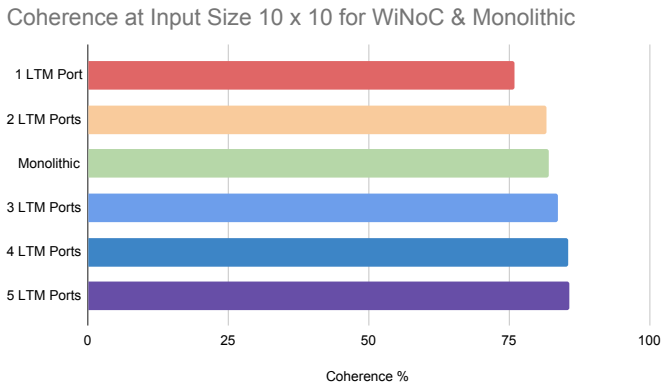



Fig. 25: Difference in Coherence for WiNoC with Varying LTM Ports and Monolithic Architecture

it also introduces new challenges that future simulators and hardware models should aim to address. Multi-core systems enable better scalability, where multiple smaller quantum cores are interconnected using both quantum and classical communication links [43][12]. This design reduces the control complexity per core and supports the execution of large-scale quantum algorithms, enabling the integration of thousands to potentially millions of qubits in a single system [44][43]. Multi-core architectures help address integration and wiring density problems, as a single monolithic chip would require impractically dense cryogenic wiring and control circuitry[29][45]. Moreover, placing qubits in separate cores has been shown to reduce interference and noise, offering a path to maintain low error rates even as the system scales [12].

However, multi-core quantum processors introduce their own set of challenges. Operations involving qubits on different cores requires transmitting quantum information or measurement outcomes between those cores, introducing a communication bottleneck absent in single-chip processors[44] [12]. This inter-core transfer incurs latency due to the additional time required for qubit state transfer and classical signaling[29]. Moreover, it demands a more complex control infrastructure, such as a network-on-chip to manage operations and control signals across the cores [12]. Additionally, these architectures must operate at cryogenic temperatures, which imposes constraints on communication bandwidth and number of wiring channels, as too many cables or active components can introduce heat and noise [45]. Ongoing research is focusing on improving quantum interconnect technology and developing intelligent qubit mapping to minimize the need for costly inter-core operations [43][29]. Future versions of multi-core simulators could benefit from modeling these trade-offs more accurately by simulating advanced routing protocols, dynamic core-to-core communication patterns, or thermal constraints to better reflect the behavior of large-scale quantum systems.

### B. *Need for In/Near-Memory Computing*

The combination of quantum computing and in-memory computing presents a promising research direction to improve system performance and enable novel architectures, such as neural networks [46][47]. This integration offers the potential for efficient algorithmic acceleration, improving computational speed and energy efficiency compared to traditional von Neumann architectures [47]. Data movement between memory and processing units has become a bottleneck, consuming time and energy [46]. Frequent back-and-forth data exchanges introduce significant latency and bandwidth limitations.

Processing-in-memory (PIM) addresses this challenge by bringing computation directly into or near the memory, reducing the need to transfer large volumes of data [46]. For instance, a 3D-stacked logic and memory architecture significantly lowers latency and power consumption while providing much higher bandwidth. Shifting toward this kind of memory-focused design could make it easier to move data efficiently in quantum systems [48][47]. Future research should explore co-designing memory and interconnect architectures for quantum systems and developing specialized in/near-memory accelerators for quantum data. These are essential to reduce latency, maximize bandwidth utilization, and improve energy efficiency in next-generation quantum computing platforms.

### C. *Energy Efficiency*

Energy efficiency is a critical concern as quantum computing systems continue to scale up. Energy consumption primarily stems from the supporting infrastructure which includes the need for a cryogenic environment, classical control electronics, and physical interconnects. These components consume significant power, which adds to the energy footprint [49]. Although modular quantum interconnects present a promising path toward scalability and greater capacity, they introduce additional energy demands associated with quantum signal generation, routing, and increased control overhead [50]. Moreover, the dense wiring used to connect control and readout electronics to cryogenic systems produces heat and imposes scalability limitations [45].

To address these challenges, energy-efficient hardware solutions, such as cryogenic control electronics and algorithms optimizations to minimize quantum-classical interactions and data movement, are required to reduce power demands [51].

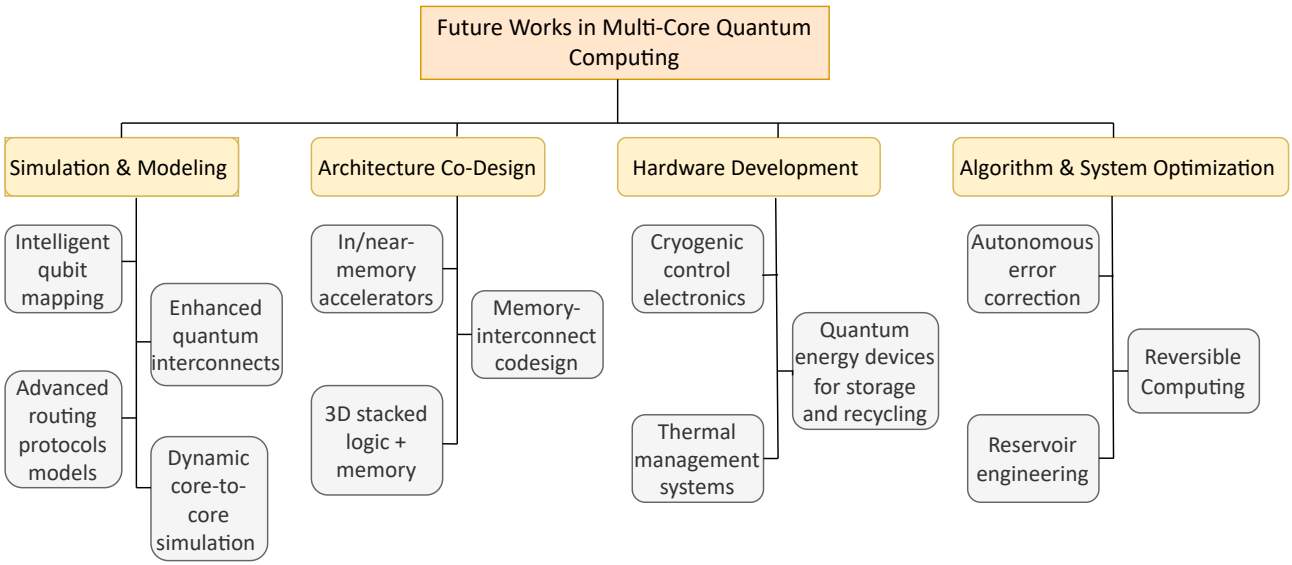



Fig. 26: Future Directions

Additional strategies for energy savings include autonomous error correction, reversible computing, and quantum energy devices for energy storage and recycling. These approaches offer the potential to reduce the energy costs associated with cryogenic operation, wire conduction, signal multiplexing, and interconnection complexity [49].

## VIII. Conclusion

This paper has effectively demonstrated the strengths and weaknesses of DQC on a 4x4 mesh architecture. It has been seen that a qNoC suffers from the same challenges as a classical NoC while also having to deal with the challenges of handling qubits. In cases where routing is slow and latency is increased, coherence can also fall off. By examining the various synthetic traffic patterns using qcomm, it was concluded that as improvements are made in DQC, it is able to efficiently handle memory intensive applications, which highlights the advantage of scalability for a distributed architecture. Seeing as the common point of contention is the problem of latency, different parametres were changed on the simulator. The two main parametres were increasing the number of LTM ports and switching to WiNoC. The above was then tested using a QCNN workload. Although monolithic still perfomred better in terms of coherence and execution time than a distributed wired NoC, after a certain input size, it was shown that the WiNoC outperforms the monolithic structure in those same areas.

To resimulate any of the data presented in this paper or to examine the various traffic generators, the repository can be found on github as "Qcomm-Traffic" [52].

Finally, while the adoption of quantum computing is limited by the number of qubits, our paper has shown that one of the most prevalent methods for increasing the number of available qubits is quite viable especially when it comes to running neural network workloads. We believe this to be especially important as we head further into the era of quantum computing.

## References

[1] DeepSeek-AI et al., “DeepSeek-V3 Technical Report,” 2024, arXiv. doi: 10.48550/arxiv.2412.19437.

[2] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” SIAM review, vol. 41, no. 2, pp. 303–332, 1999, doi: 10.1137/s0036144598347011

[3] L. K. Grover, “A fast quantum mechanical algorithm for database search,” 1996, doi: 10.48550/arxiv.quant-ph/9605043

[4] Sangari, Mohamad Sadegh; Lee, Michael; and Mashatan, Atefeh, "TOWARDS QUANTUM THREAT MITIGATION: AN EMPIRICAL INVESTIGATION OF THE FACTORS INFLUENCING ORGANIZATIONAL PREPARATION INTENTIONS" (2025). ECIS 2025 Proceedings. 1. https://aisel.aisnet.org/ecis2025/security/security/1

[5] Y. Liao and C. Ferrie, “GPT on a Quantum Computer,” 2024, doi: 10.48550/arxiv.2403.09418

[6] C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” Quantum, vol. 5, p. 433, Apr. 2021. doi:10.22331/q-2021-04-15-433

[7] J. Gambetta. "The hardware and software for the era of quantum utility is here". ibm.com. Accessed: Aug. 13, 2025. [Online] Available: https://www.ibm.com/quantum/blog/quantum-roadmap-2033

[8] J. Pan, “Effective quantum error correction by AI,”

Nature Computational Science, vol. 4, no. 12, pp. 881–881, 2024, doi: 10.1038/s43588-024-00755-9

[9] A. Singh, K. Dev, H. Siljak, H. D. Joshi, and M. Magarini, "Quantum Internet-Applications, Functionalities, Enabling Technologies, Challenges, and Research Directions," IEEE Communications surveys and tutorials, vol. 23, no. 4, pp. 2218–2247, 2021, doi: 10.1109/COMST.2021.3109944

[10] S. Al Abdul Wahid, A. Asad, and F. Mohammadi, "A Survey on Neuromorphic Architectures for Running Artificial Intelligence Algorithms," Electronics (Basel), vol. 13, no. 15, p. 2963, 2024, doi: 10.3390/electronics13152963

[11] R. Van Meter and S. J. Devitt, "The Path to Scalable Distributed Quantum Computing," Computer (Long Beach, Calif.), vol. 49, no. 9, pp. 31–42, 2016, doi: 10.1109/MC.2016.291

[12] M. Palesi, E. Russo, D. Patti, G. Ascia, and V. Catania, "Assessing the Role of Communication in Scalable Multi-Core Quantum Architectures," in Proceedings (IEEE International Symposium on Embedded Multicore/Manycore SoCs. Online), IEEE, 2024, pp. 482–489. doi: 10.1109/MCSoC64144.2024.00085

[13] Y. Solihin, Fundamentals of parallel multicore architecture, 1st edition. Boca Raton, FL: Chapman and Hall/CRC, an imprint of Taylor and Francis, 2015

[14] R. S. P. S, R. Gupta, M. Palesi, and J. Jose, "Decentralized Framework for Teleportation in Quantum Core Interconnects," 2025, doi: 10.48550/arxiv.2505.11195

[15] S. Lloyd, M. Mohseni, and P. Rebentrost, "Quantum principal component analysis," Nature physics, vol. 10, no. 9, pp. 631–633, 2014, doi: 10.1038/nphys3029

[16] M. Caleffi, M. Amoretti, D. Ferrari, J. Illiano, A. Manzalini, and A. S. Cacciapuoti, "Distributed quantum computing: A survey," Computer networks (Amsterdam, Netherlands : 1999), vol. 254, Art. no. 110672, 2024, doi: 10.1016/j.comnet.2024.110672

[17] S. Wang and T. Jin, "Wireless network-on-chip: a survey," Journal of engineering (Stevenage, England), vol. 2014, no. 3, pp. 98–104, 2014, doi: 10.1049/joe.2013.0209

[18] A. S. Cacciapuoti, M. Caleffi, R. Van Meter, and L. Hanzo, "When Entanglement Meets Classical Communications: Quantum Teleportation for the Quantum Internet," IEEE transactions on communications, vol. 68, no. 6, pp. 3808–3833, 2020, doi: 10.1109/TCOMM.2020.2978071

[19] S. D. Barrett and P. Kok, "Efficient high-fidelity quantum computation using matter qubits and linear optics," Physical review. A, Atomic, molecular, and optical physics, vol. 71, no. 6, Art. no. 060310, 2005, doi: 10.1103/PhysRevA.71.060310

[20] J. Peckham, D. Makaroff, and S. Rayan, "Asynchronous Telegate and Teledata Protocols for Distributed Quantum Computing," 2024, doi: 10.48550/arxiv.2407.14987

[21] V. Rajesh, U. P. Naik, and Mohana, "Quantum Convolutional Neural Networks (QCNN) Using Deep Learning for Computer Vision Applications," in 2021 International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT), IEEE, 2021, pp. 728–734. doi: 10.1109/RTEICT52294.2021.9574030

[22] K. O'Shea and R. Nash, "An Introduction to Convolutional Neural Networks," 2015, doi: 10.48550/arxiv.1511.08458

[23] S. Indolia, A. K. Goswami, S. P. Mishra, and P. Asopa, "Conceptual Understanding of Convolutional Neural Network-A Deep Learning Approach," Procedia computer science, vol. 132, pp. 679–688, 2018, doi: 10.1016/j.procs.2018.05.069

[24] The Quantum Convolution Neural Network GITHUB CITE **This source is ambiguous to cite**

[25] N. Munikote, "Comparing Quantum Encoding Techniques," 2024, doi: 10.48550/arxiv.2410.09121

[26] M. Rath and H. Date, "Quantum data encoding: a comparative analysis of classical-to-quantum mapping techniques and their impact on machine learning accuracy," EPJ quantum technology, vol. 11, no. 1, Art. no. 72, 2024, doi: 10.1140/epjqt/s40507-024-00285-3

[27] A. Das, M. Palesi, J. Kim, and P. Pratim Pande, "Chip and Package-Scale Interconnects for General-Purpose, Domain-Specific, and Quantum Computing Systems-Overview, Challenges, and Opportunities," IEEE journal on emerging and selected topics in circuits and systems, vol. 14, no. 3, pp. 354–370, 2024, doi: 10.1109/JETCAS.2024.3445829

[28] J. Niu et al., "Low-loss interconnects for modular superconducting quantum processors," Nature electronics, vol. 6, no. 3, pp. 235–241, 2023, doi: 10.1038/s41928-023-00925-z

[29] E. Alarcon et al., "Scalable multi-chip quantum architectures enabled by cryogenic hybrid wireless/quantum-coherent network-in-package," in 2023 IEEE International Symposium on Circuits and Systems (ISCAS), IEEE, 2023, pp. 1–5. doi: 10.1109/ISCAS46773.2023.10181857

[30] S. Rodrigo, S. Abadal, C. G. Almud´ever, and E. Alarc´on, "Modelling short-range quantum teleportation for scalable multi-core quantum computing architectures," in Proceedings of the Eight Annual ACM International Conference on Nanoscale Computing and Communication, 2021.

[31] I. Cong, S. Choi, and M. D. Lukin, "Quantum

convolutional neural networks," Nature physics, vol. 15, no. 12, pp. 1273–1278, 2019, doi: 10.1038/s41567-019-0648-8

[32] Q. Wu, W. Liu, Y. Huang, H. Liu, H. Xiao, and Z. Li, "A degressive quantum convolutional neural network for quantum state classification and code recognition," iScience, vol. 27, no. 4, Art. no. 109394, 2024, doi: 10.1016/j.isci.2024.109394

[33] L.-H. Gong, J.-J. Pei, T.-F. Zhang, and N.-R. Zhou, "Quantum convolutional neural network based on variational quantum circuits," Optics communications, vol. 550, Art. no. 129993, 2024, doi: 10.1016/j.optcom.2023.129993

[34] E. Ovalle-Magallanes, D. E. Alvarado-Carrillo, J. G. Avina-Cervantes, I. Cruz-Aceves, and J. Ruiz-Pinales, "Quantum angle encoding with learnable rotation applied to quantum–classical convolutional neural networks," Applied soft computing, vol. 141, Art. no. 110307, 2023, doi: 10.1016/j.asoc.2023.110307

[35] R. Kaur, A. Asad, S. Al Abdul Wahid, and F. Mohammadi, "A Survey of Advancements in Scheduling Techniques for Efficient Deep Learning Computations on GPUs," Electronics (Basel), vol. 14, no. 5, p. 1048, 2025, doi: 10.3390/electronics14051048

[36] R. Kaur, A. Asad, and F. Mohammadi, "A Comprehensive Review of Processing-in-Memory Architectures for Deep Neural Networks," Computers (Basel), vol. 13, no. 7, p. 174, 2024, doi: 10.3390/computers13070174

[37] A. Asad, R. Kaur, and F. Mohammadi, "A Survey on Memory Subsystems for Deep Neural Network Accelerators," Future internet, vol. 14, no. 5, p. 146, 2022, doi: 10.3390/fi14050146

[38] K. Beer et al., "Training deep quantum neural networks," Nature communications, vol. 11, no. 1, Art. no. 808, 2020, doi: 10.1038/s41467-020-14454-2

[39] S. D. Chawade, M. A. Gaikwad, and R. M. Patrikar, "Review of XY Routing Algorithm for Network-on-Chip Architecture," International journal of computer applications, vol. 43, no. 21, 2012, doi: 10.5120/6387-8691

[40] M. A. Shafique, N. K. Baloch, M. I. Baig, F. Hussain, Y. B. Zikria, and S. W. Kim, "NoCGuard: A Reliable Network-on-Chip Router Architecture," Electronics (Basel), vol. 9, no. 2, p. 342, 2020, doi: 10.3390/electronics9020342

[41] S. A. Abdul Wahid, A. Asad, R. Kaur, and F. Mohammadi, "Quantum Computing Circuit Design: A Tutorial," in 2024 International Conference on Advanced Scientific Computing (ICASC), IEEE, 2024, pp. 1–6. doi: 10.1109/ICASC63229.2024.10785069

[42] A. Lit, S. Suhaili, K. Kipli, and N. Rajaee, "Performance Analysis of NoC and WiNoC in Multicore System Architectures," The International journal of networked and distributed computing (Online), vol. 13, no. 1, Art. no. 13, 2025, doi: 10.1007/s44227-024-00046-9

[43] A. Ovide et al., "Mapping quantum algorithms to multi-core quantum computing architectures," 2023, doi: 10.48550/arxiv.2303.16125

[44] S. B. Rached et al., "Characterizing the Inter-Core Qubit Traffic in Large-Scale Quantum Modular Architectures," IEEE access, vol. 13, pp. 113236–113257, 2025, doi: 10.1109/ACCESS.2025.3583218

[45] P. Escofet, S. B. Rached, S. Rodrigo, C. G. Almudever, E. Alarcón, and S. Abadal, "Interconnect Fabrics for Multi-Core Quantum Processors: A Context Analysis," 2023, doi: 10.48550/arxiv.2309.07313

[46] K. Asifuzzaman, N. R. Miniskar, A. R. Young, F. Liu, and J. S. Vetter, "A survey on processing-in-memory techniques: Advances and challenges," Memories - Materials, Devices, Circuits and Systems, vol. 4, Art. no. 100022, 2023, doi: 10.1016/j.memori.2022.100022

[47] Z. Miao, Y. Li, S. Wang, and Z. Sun, "Hybrid quantum and in-memory computing for accelerating solving Simon's problem," APL machine learning, vol. 3, no. 2, pp. 026117-026117–15, 2025, doi: 10.1063/5.0268483

[48] Pratibha and N. Mahmud, "A Reconfigurable Framework for Hybrid Quantum–Classical Computing," Algorithms, vol. 18, no. 5, p. 271, 2025, doi: 10.3390/a18050271

[49] A. Auffèves, "Quantum Technologies Need a Quantum Energy Initiative," PRX quantum, vol. 3, no. 2, Art. no. 020101, 2022, doi: 10.1103/PRXQuantum.3.020101

[50] On the Impact of Classical and Quantum Communication Networks Upon Modular Quantum Computing Architecture System Performance

[51] J. C. Brennan et al., "Classical Interfaces for Controlling Cryogenic Quantum Computing Technologies," 2025, doi: 10.48550/arxiv.2504.18527 Technologies

[52] S. Joarder et al., "Qcomm-Traffic" 2025. [Online]. Available: https://github.com/Sadit-J/Qcomm-Traffic